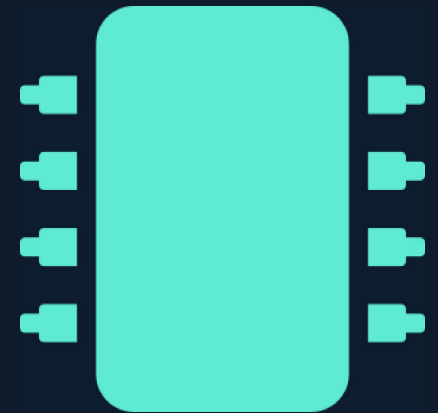


FACULTY ORIENTATION PROGRAM ON VLSI DESIGN

Introduction to Verilog HDL

Concepts, Design Flow, Abstraction Levels & Hierarchical Modeling



Dr. Kanchan S. Tiwari

Department of Electronics & Telecommunication Engineering

MES Wadia College of Engineering, Pune



Session Roadmap

1. Verilog Fundamentals

- ◆ Introduction to Verilog HDL
- ◆ Design flow
- ◆ Top-down vs Bottom-up methodology

2. Module Structure

- ◆ Module structure & ports
- ◆ Design hierarchy
- ◆ Declaration & instantiation

3. Levels of Abstraction

- ◆ Behavioral
- ◆ Dataflow
- ◆ Gate level
- ◆ Switch level

4. Language Support

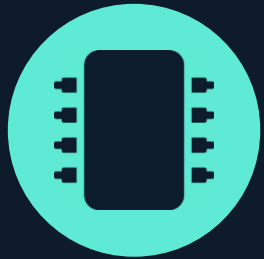
- ◆ Compiler directives
- ◆ System tasks & functions

5. Verification Flow

- ◆ Simulation vs synthesis
- ◆ Testbenches
- ◆ File organization

6. Hands-on Example

- ◆ Hierarchical modeling walkthrough



SECTION 1 OF 5

Verilog Fundamentals

What Verilog HDL is, the design flow, and top-down / bottom-up methodology



Introduction: Why Do We Need CAD Tools?

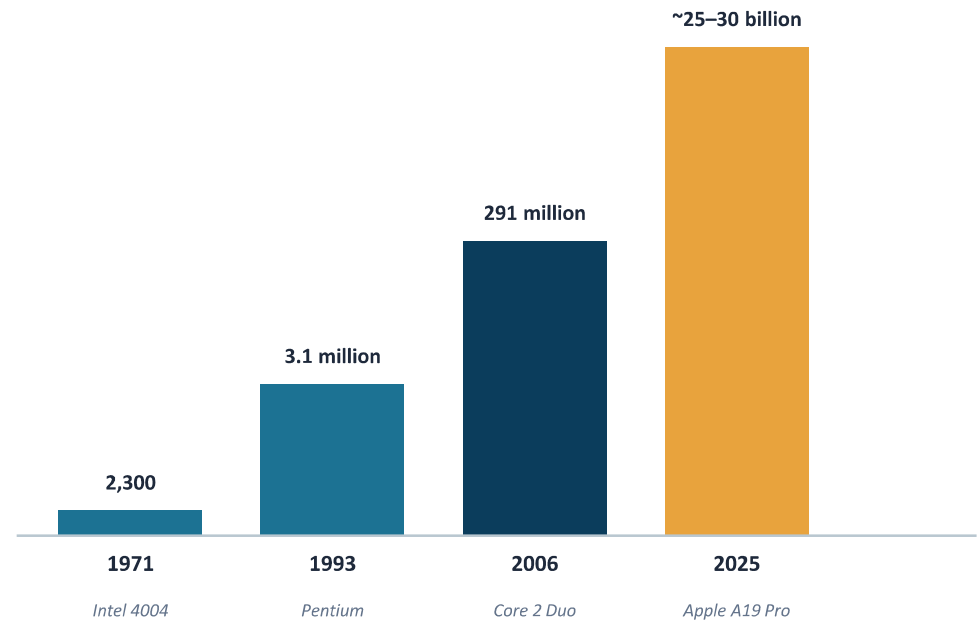
- ✦ Early digital circuits were designed by hand — drawing gate-level schematics directly.
- ✦ As circuits grew from tens to millions of gates, schematic capture became:

- Too complex to draw and manage
- Prone to human error
- Extremely time-consuming to simulate & debug

Solution → Computer Aided Design (CAD) tools, based on Hardware Description Languages (Verilog, VHDL) — describe the circuit in text, let software simulate & synthesize it.

Moore's Law in Action

Transistor count per chip, over time





REAL-WORLD SCALE

Apple A19 Pro — the chip in the iPhone 17 Pro (2025)



~25–30 Billion

Transistor Count

vs. 2,300 in the 1971 Intel 4004



3 nm-class

Feature Size

TSMC “N3P” process node



< 1 V (typ.)

Core Supply Voltage

battery supplies ~3.7–4.5 V system-side

Billions of transistors, verified and connected correctly — by hand-drawn schematics, this would be simply impossible. This is exactly why CAD tools and HDLs like Verilog exist.

Hardware Description Language (HDL)

- HDLs are specialized computer languages used to program electronic and digital logic circuits.
- HDL is a computer aided design (CAD) tool for the modern digital design and synthesis of digital systems.
- High level languages with which we can specify our HW to analyze its design before actual fabrication.
- Two most popular HDLs:
 - Verilog
 - VHDL
- Other popular HDLs:
 - SystemC
 - SystemVerilog
 - ...



A Tale of Two HDLs: How Verilog & VHDL Were Born



VHDL

Born Top-Down — by government mandate

1980

US Dept. of Defense launches the VHSIC program — chips are getting too complex for every vendor to document differently.

1983

DoD awards a contract to a consortium: Intermetrics, IBM & Texas Instruments — built by committee, from a spec.

1985

VHDL v7.2 released — based on Ada, verbose and rigorous by design.

1987

Becomes IEEE Standard 1076. DoD contractors are essentially required to use it.



Verilog

Born Bottom-Up — by one startup's need

1983

Phil Moorby joins tiny startup Gateway Design Automation, needing a faster logic simulator to sell.

'83/'84

Over one winter, Moorby personally designs Verilog in about a month — just to drive that one simulator.

1985

Verilog-XL ships and is adopted fast by AMD, Sun, Hughes Aircraft — popular purely on merit, not mandate.

1989–90

Cadence Design Systems acquires Gateway. Verilog is still a closed, proprietary trade secret.



THE TWIST

How a Proprietary Tool Became a Global Standard

1

By 1990, VHDL had the DoD's mandate AND an open IEEE standard (1076). Verilog was fast and popular — but still a closed, proprietary Cadence product.

2

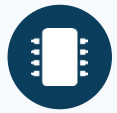
An open standard usually beats a closed tool in the long run — so Cadence made a bold move: they released Verilog into the public domain in 1990 (“Open Verilog International”), giving away their competitive secret to survive.

3

It worked. Verilog became IEEE Standard 1364 in 1995 — a startup's proprietary trade secret and a government committee's mandated language, now sitting side by side as co-equal industry standards.



Sound familiar? This is the same top-down vs. bottom-up contrast from Section 3 — except here it's not a design technique, it's the reason the language you're learning exists. Both are alive today: Verilog grew into SystemVerilog (IEEE 1800); VHDL kept its own revisions through 2019.



Introduction to Verilog HDL

- ◆ Verilog is a Hardware Description Language (HDL) used to model, design, verify and document digital electronic systems.
- ◆ Standardized as IEEE 1364; describes hardware at register-transfer level (RTL), gate level and switch level.
- ◆ Unlike C/C++, Verilog statements execute concurrently — it models real hardware where many operations happen simultaneously.
- ◆ Used across the complete digital design flow: specification → RTL coding → simulation → synthesis → fabrication.
- ◆ Widely adopted in industry (ASIC/FPGA design) and academia; taught as the entry point to digital VLSI design.

Evolution of Verilog

- ◆ In 1983, a company called Gateway design Automation developed a hardware- description language for its newly introduced logic simulator verilog_XL
- ◆ Gateway was bought by cadence in 1989 & cadence made Verilog available as public domain.
- ◆ In December 1995, Verilog HDL became IEEE standard 1364-1995.
- ◆ The language presently is maintained by the Open Verilog international (OVI) organization.
- ◆ Verilog code structure is based on C software language.



Basic Verilog Design Flow

1. Design Specification

- Abstract description of functionality, interface, and architecture
- Describes *what* the circuit should do — not *how* to build it yet

2. Behavioral Description

- High-level model to validate functionality, performance, and standards compliance.
- Often written in HDL, or in C/C++-based modeling tools
- No gate-level or timing detail yet — purely "does the idea work?"

3. RTL Description (HDL)

- Behavioral model manually rewritten as data flow between registers via combinational logic
- The most human-effort-intensive step — the "sweet spot" of abstraction
- Design is considered "frozen" here; everything after this is EDA-tool assisted
- Has a self-loop in the flow — RTL is refined/re-edited as bugs are found downstream.

4. Functional Verification and Testing

- RTL simulated against testbenches to confirm correct behavior
- Bugs found here loop back to RTL for correction.

5. Logic Synthesis / Timing Verification

- EDA tool automatically converts RTL → gate-level netlist
- Checks that timing, area, and power specifications are met.

6. Gate-Level Netlist

- Purely structural description: gates + interconnections
- No more behavioral/RTL abstraction left

7. Logical Verification and Testing

- Confirms gate-level netlist is functionally equivalent to the RTL it came from
- Failures can loop back to RTL Description

8. Floor Planning, Automatic Place and Route

- Decides physical placement of blocks and routes interconnect wiring on the die

9. Physical Layout

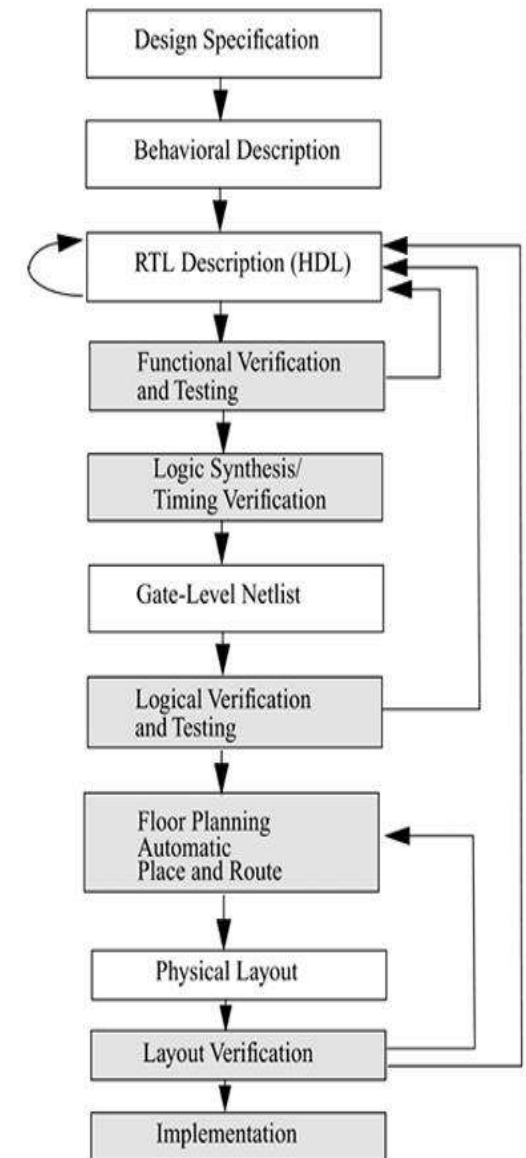
- Final geometric representation (transistors, wells, metal layers) ready for fabrication

10. Layout Verification

- DRC (Design Rule Check) — layout obeys foundry manufacturing rules
- LVS (Layout Versus Schematic) — layout matches the netlist
- Failures can loop back to Floor Planning or even RTL

11. Implementation

- Final fabrication on chip, after all verification loops close cleanly

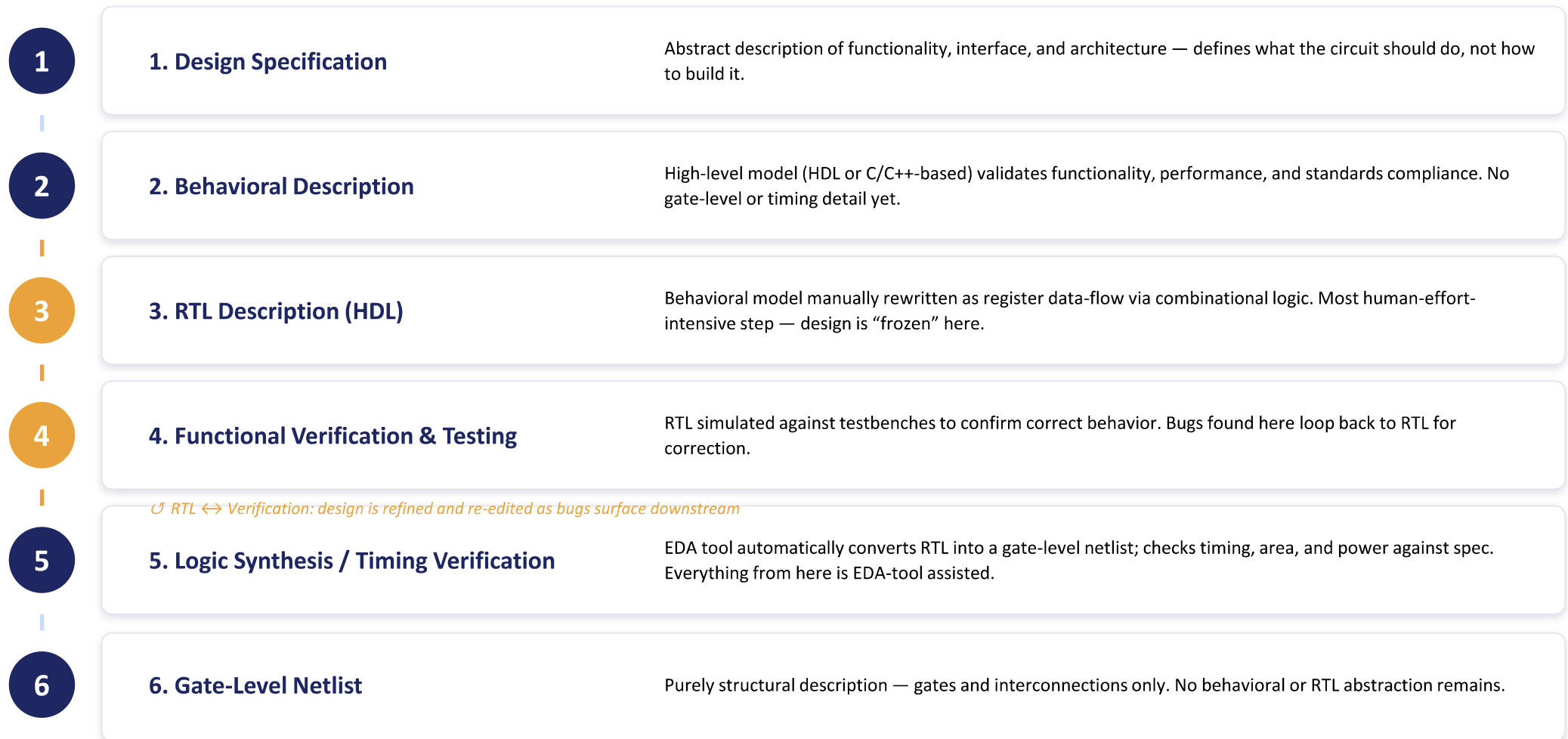


Key Takeaways

- Flow is **iterative, not linear** — every verification stage can send the designer back to an earlier stage (usually RTL)
- RTL absorbs almost all manual design effort; everything before it is exploratory, everything after it is automated
- RTL-based design (vs. old schematic capture) shrank design cycles from **years → months**
- EDA tools automate synthesis/layout but don't replace judgment — poor RTL still produces a poor chip ("**Garbage In, Garbage Out**")
- Emerging **behavioral/high-level synthesis** tools generate RTL automatically from algorithmic descriptions — pushing automation one level higher

Specification to Gate-Level Netlist

11-Stage Flow from Abstract Specification to Fabricated Silicon



Gate-Level Netlist to Fabrication

11-Stage Flow from Abstract Specification to Fabricated Silicon



↪ Verification loops (Steps 7 & 10) can send the design all the way back to RTL before final sign-off



Top-Down Design Methodology

System Specification

Architectural / Behavioral Model

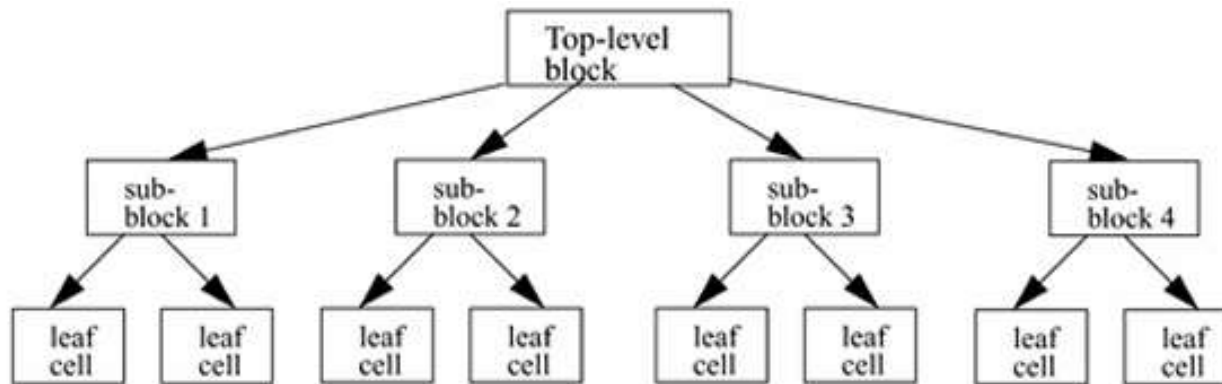
RTL (Register-Transfer Level)

Gate-Level Implementation

Key idea

- ◆ Design starts from the overall system view and is progressively refined into detailed sub-modules.
- ◆ Each level is verified before descending to the next.
- ◆ Encourages early architectural exploration and clear specification before implementation details.

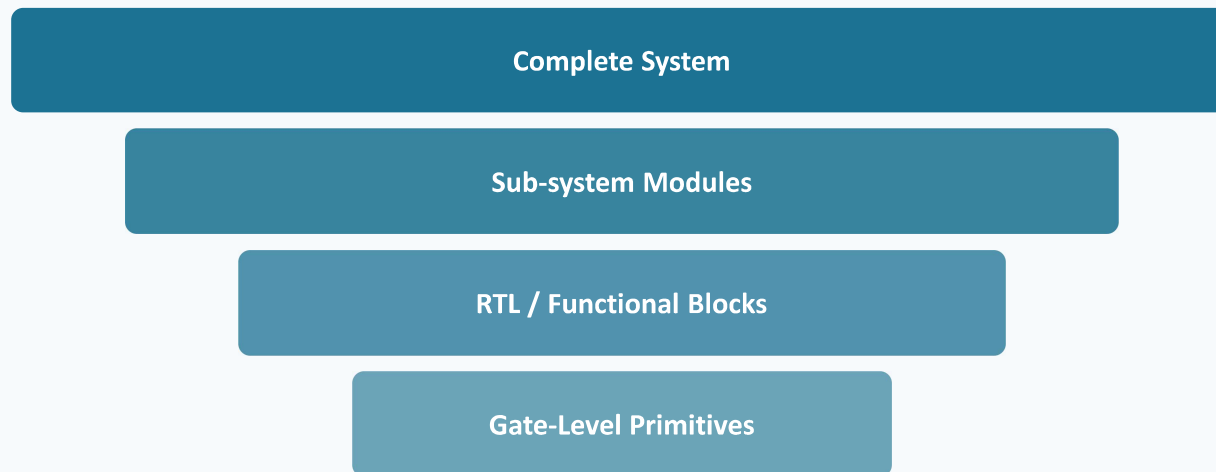
Top-Down Design Methodology



In a top-down design methodology, we define the top-level block and identify the sub-blocks necessary to build the top-level block. We further subdivide the sub-blocks until we come to leaf cells, which are the cells that cannot further be divided.

↑ Bottom-Up Design Methodology

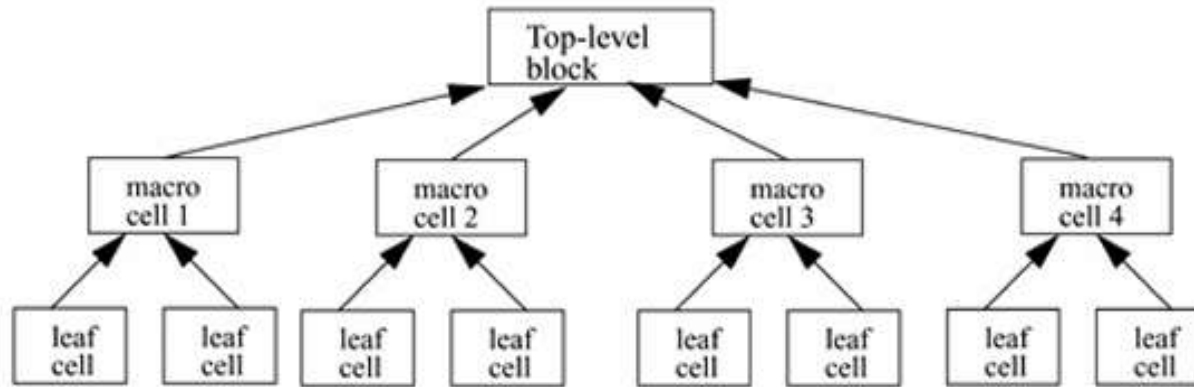
Existing verified building blocks (gates, cells, IP) are combined and integrated upward into larger functional units and finally the complete system.



Top-Down vs Bottom-Up

Aspect	Top-Down	Bottom-Up
Start point	System spec	Existing blocks
Direction	Refine downward	Integrate upward
Reuse	Lower	Higher
Common use	New architectures	Reusing IP / cells

Bottom-Up Design Methodology



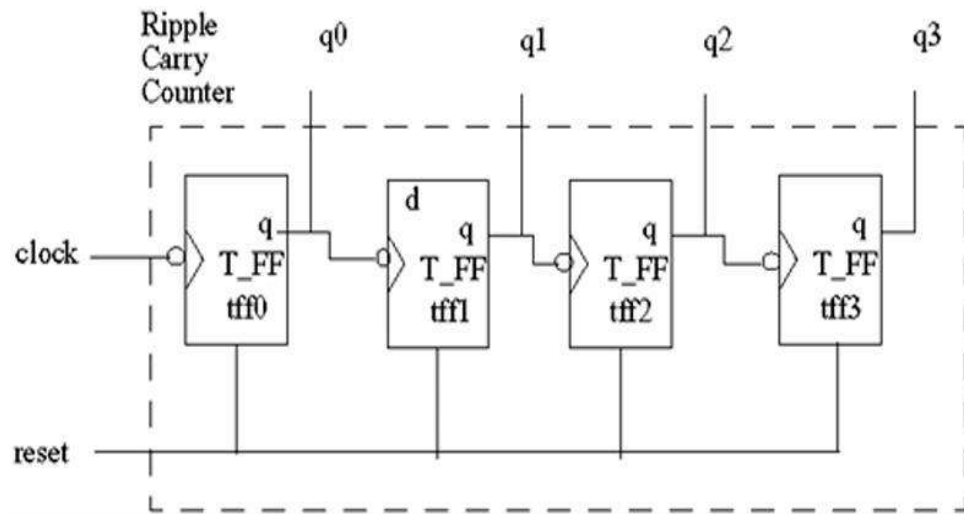
In a bottom-up design methodology, we first identify the building blocks that are available to us. We build bigger cells, using these building blocks. These cells are then used for higher-level blocks until we build the top-level block in the design.

VLSI Design Flow: Top-Down and Bottom-Up Approach

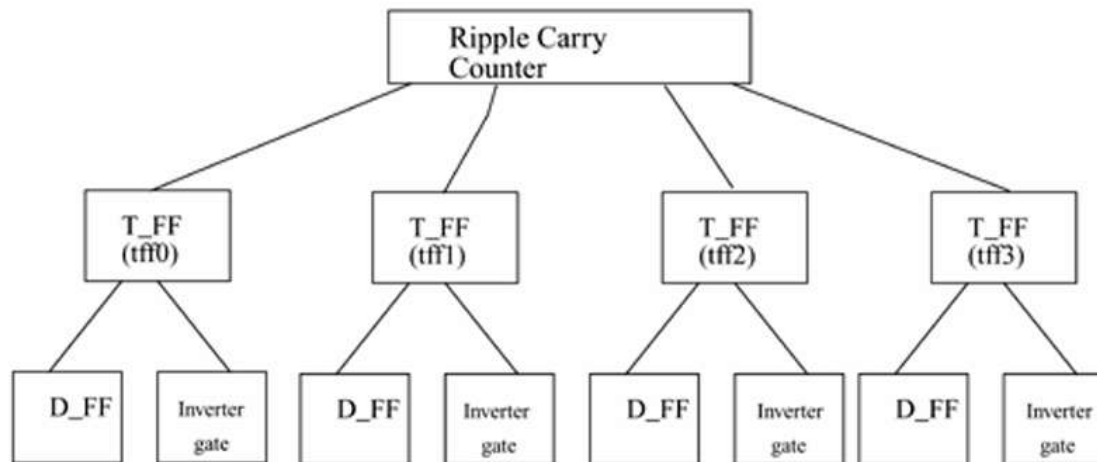
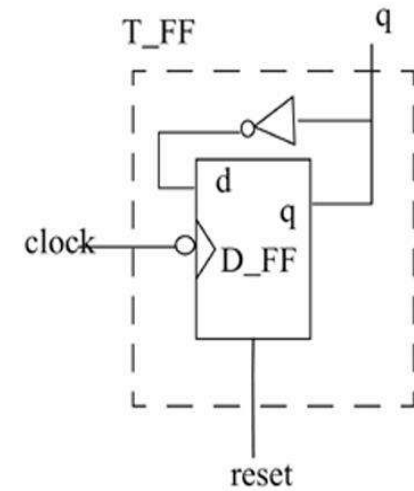
A combination of top-down and bottom-up flows is typically used.

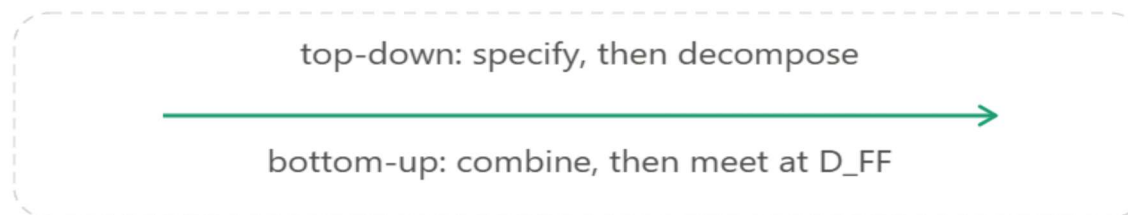
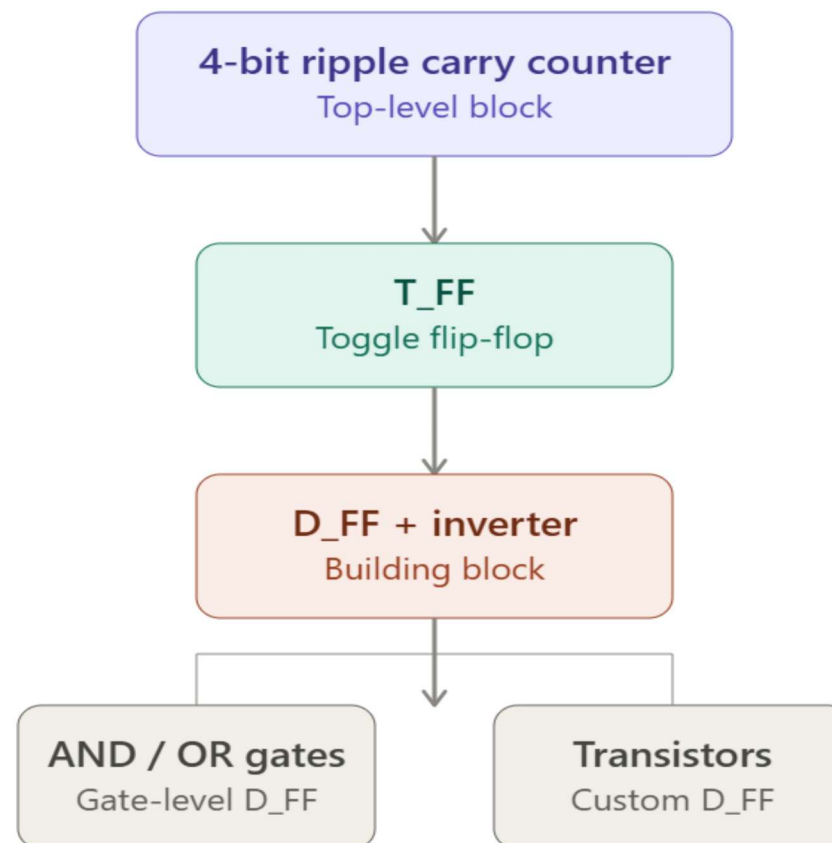
- Design architects:** define specifications of the top-level block
- Logic designers:** structure the design by breaking functionality into blocks and sub-blocks
- Circuit designers:** simultaneously design optimized circuits for leaf-level cells
 - Build higher-level cells using these leaf cells
- Meeting point:** the two flows converge at an intermediate level
 - Switch-level designers create a library of leaf cells using switches
 - Logic-level designers work top-down until all modules are defined in terms of leaf cells

4 bit Ripple Carry Counter



reset	q_n	q_{n+1}
1	1	0
1	0	0
0	0	1
0	1	0







SECTION 2 OF 5

Module Structure & Hierarchy

How a Verilog module is written, its ports, and how designs are organized hierarchically



Structure of a Verilog Module

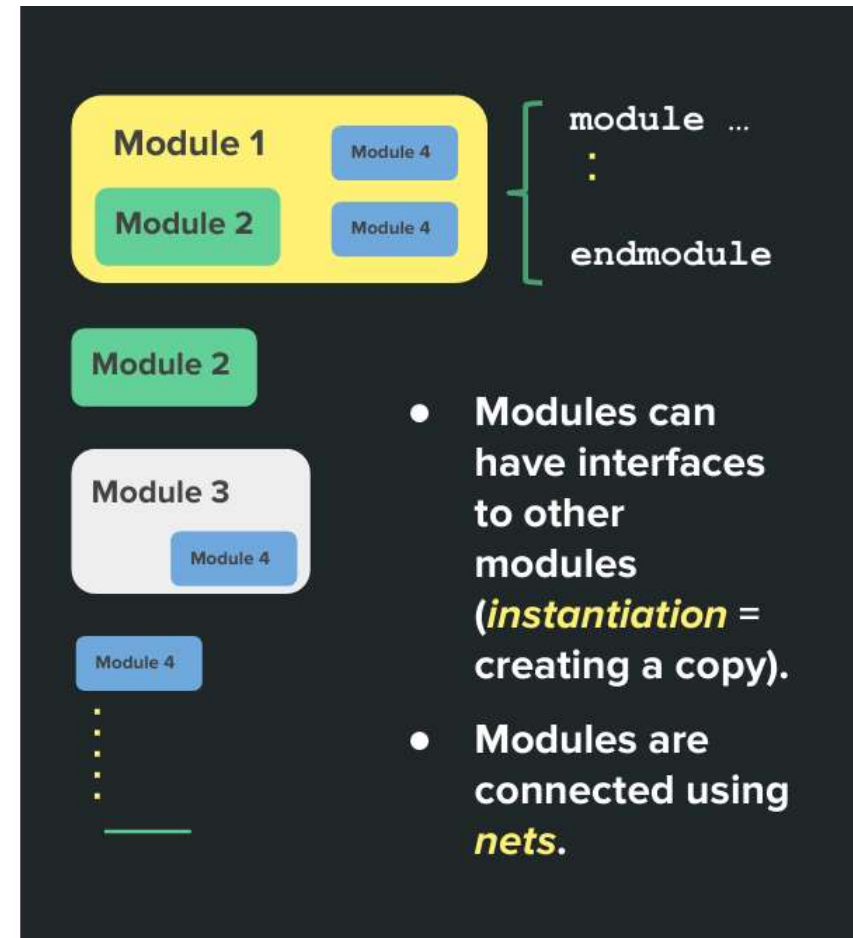
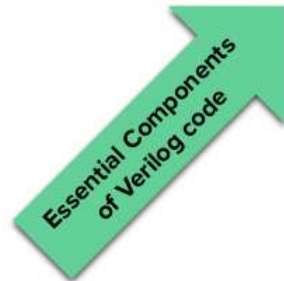
```
module and_gate (out, a, b);    // module name +  
port list  
  
    output out;                // port  
declarations  
    input  a, b;  
  
    wire out;                  // internal signal  
declarations  
  
    and (out, a, b);           // design body:  
                                // behavior /  
structure  
  
endmodule                      // end of module
```

- ✦ Verilog provides the concept of a module. A module is the basic building block in Verilog.
- ✦ module ... endmodule: every design unit is enclosed between these keywords.
- ✦ Port list: interface signals through which the module connects to the outside world.
- ✦ Port declarations: specify direction (input / output / inout) for each port.
- ✦ Internal declarations: wire and reg signals used inside the module.
- ✦ Design body: contains the actual behavior or structure — gate instances, continuous assignments, or always blocks.
- ✦ A module can be instantiated any number of times inside other modules.

Getting Started

With Verilog

Describing a digital system as a
set of modules





Port Declarations: input, output, inout

input

Brings a signal into the module. Must be declared as a net (wire) type — driven by the calling environment.

```
input a, b;
```

output

Drives a signal out of the module. Can be declared as wire or reg (reg when assigned inside an always block).

```
output reg y;
```

inout

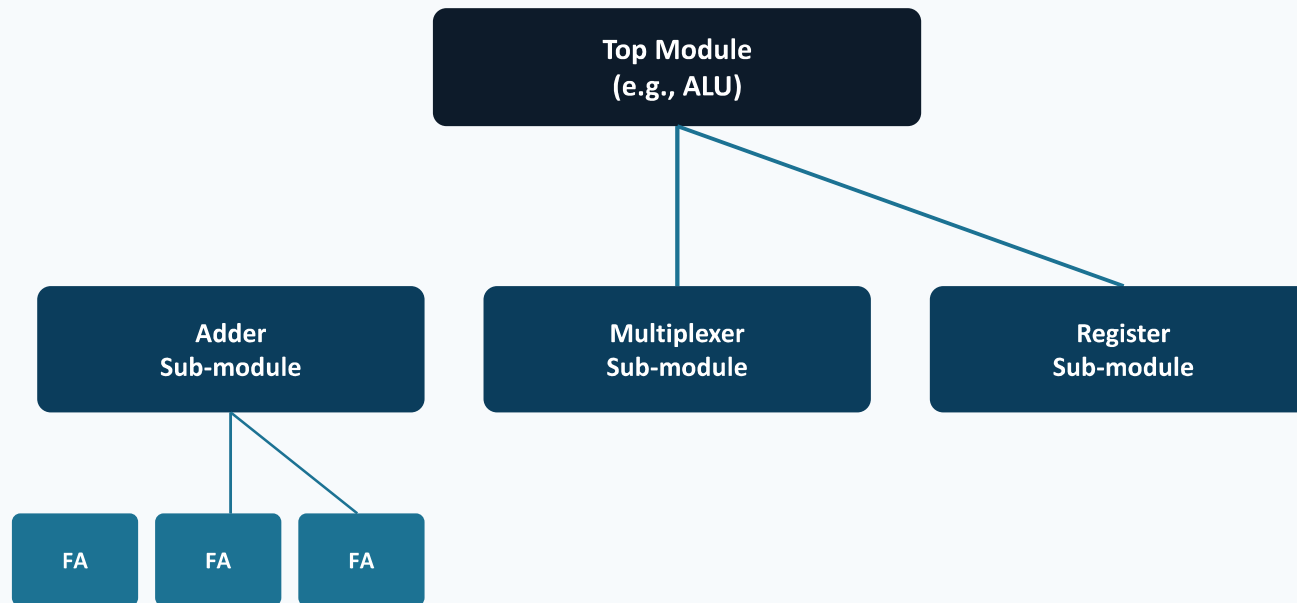
Bidirectional port — can act as both input and output, commonly used for buses and tri-state connections.

```
inout data_bus;
```

- ✦ Port direction and width must match between a module's declaration and the way it is connected at instantiation.
- ✦ In Verilog-2001+ style, ports can be declared with direction and data type together directly in the port list, e.g. module dff(output reg q, input d, clk);



Design Hierarchy Concepts



- ✦ A design is organized as a hierarchy: a top-level module instantiates lower-level sub-modules, which may themselves instantiate further modules.
- ✦ Hierarchy promotes modularity, reusability, parallel team development, and easier debugging / verification of each block in isolation.



Module Declaration & Instantiation

```
// Module declaration
module half_adder (sum, carry, a, b);
    output sum, carry;
    input  a, b;
    assign sum    = a ^ b;
    assign carry  = a & b;
endmodule
```

```
// Instantiation inside a parent module
module full_adder (sum, cout, a, b, cin);
    output sum, cout;
    input  a, b, cin;
    wire w1, w2, w3;
    half_adder HA1 (w1, w2, a, b);    // instance 1
    half_adder HA2 (sum, w3, w1, cin); // instance 2
    or (cout, w2, w3);
endmodule
```

- ◆ Declaration defines a module's name, ports and internal behavior once.
- ◆ Instantiation creates a physical copy (instance) of that module inside another module, each with its own unique instance name (e.g. HA1, HA2).
- ◆ Connections can use positional mapping (order of ports) or named mapping, e.g. .a(x), .b(y) — named mapping is safer for large port lists.
- ◆ A module can be instantiated multiple times, enabling reuse — the basis of hierarchical, bottom-up design.



SECTION 3 OF 5

Levels of Abstraction

Behavioral, Dataflow, Gate and Switch level modeling styles in Verilog



Four Levels of Abstraction

Level	Description	Key Constructs
Behavioral	Describes what the circuit does using algorithm-like statements; closest to software	always, initial, if-else, case
Dataflow	Describes how data flows between registers via continuous logic expressions	assign, operators (&, , ^, +)
Gate	Circuit is built from interconnected logic gate primitives	and, or, nand, nor, xor, not
Switch	Lowest level; models transistors (MOS switches) directly	nmos, pmos, cmos, tran

- ✦ A single design can freely mix multiple abstraction levels (mixed-level modeling) within the same hierarchy.
- ✦ Higher levels (behavioral) are easier to write and verify; lower levels (gate / switch) map closer to real silicon.



Behavioral Level Modeling

```
module mux2_1 (out, i0, i1, sel);  
    output reg out;  
    input  i0, i1, sel;  
  
    always @ (i0 or i1 or sel) begin  
        if (sel == 1'b0)  
            out = i0;  
        else  
            out = i1;  
        end  
    endmodule
```

- ✦ Describes circuit behavior algorithmically — similar in style to a programming language.
- ✦ Uses procedural blocks: always (for combinational/sequential logic) and initial (for one-time setup, mainly in testbenches).
- ✦ Outputs assigned inside procedural blocks must be declared as reg (they hold a value between updates).
- ✦ Best suited for describing complex control logic such as FSMs, ALUs, and processors before structural refinement.



Dataflow Level Modeling

```
module mux2_1 (out, i0, i1, sel);  
    output out;  
    input  i0, i1, sel;  
  
    assign out = (~sel & i0) | (sel & i1);  
endmodule
```

- ✦ Describes a circuit in terms of continuous assignments — how data 'flows' from inputs to outputs through logical/arithmetic expressions.
- ✦ Uses the assign keyword; the right-hand side is evaluated continuously whenever an operand changes.
- ✦ Target signal must be declared as wire, not reg.
- ✦ Sits between behavioral and gate level — more structural than behavioral, more abstract than gate-level netlists.



Gate Level Modeling

```
module mux2_1 (out, i0, i1, sel);  
    output out;  
    input  i0, i1, sel;  
    wire sel_n, w1, w2;  
  
    not (sel_n, sel);  
    and (w1, i0, sel_n);  
    and (w2, i1, sel);  
    or  (out, w1, w2);  
endmodule
```

- ✦ Circuit described as an interconnection of built-in logic gate primitives: and, or, not, nand, nor, xor, xnor, buf.
- ✦ Each gate instance connects to signals declared as wire.
- ✦ Closely resembles a schematic / logic diagram — useful for teaching Boolean-to-hardware mapping.
- ✦ This is typically the output style produced automatically by a synthesis tool from RTL code.



Switch Level Modeling

```
module cmos_inverter (out, in);  
    output out;  
    input in;  
    supply1 vdd;  
    supply0 gnd;  
  
    pmos p1 (out, vdd, in);  
    nmos n1 (out, gnd, in);  
endmodule
```

- ✦ The lowest and most detailed abstraction level — models individual MOS transistors as switches.
- ✦ Key primitives: nmos, pmos, cmos, and bidirectional pass switches (tran, tranif0/1).
- ✦ Used to describe transistor-level circuits such as CMOS gates, pass-transistor logic, and transmission gates.
- ✦ Rarely written by hand for large designs today, but essential for understanding CMOS process technology and custom cell design.

Summary

Abstraction Level vs. Design Flexibility

- Higher levels of abstraction → more flexible, technology-independent designs
- Lower levels (toward switch-level) → design becomes technology-dependent and inflexible
- A small modification at a lower level can trigger significant cascading changes

Analogy: C vs. Assembly Programming

- Higher-level languages (e.g., C) are easier to program in
- C programs are easily portable across machines
- Assembly-level programs are machine-specific.
- Assembly code cannot be easily ported to another machine



SECTION 4 OF 5

Compiler Directives & System Tasks

Preprocessor-style directives and built-in tasks/functions used across Verilog code

>_ Compiler Directives

Directive	Purpose
<code>`define</code>	Create a text macro / constant, e.g. <code>`define WIDTH 8</code>
<code>`include</code>	Insert contents of another Verilog file at compile time typically header files containing global or commonly used definitions
<code>`timescale</code>	Set simulation time unit and precision, e.g. <code>`timescale 1ns/1ps</code>
<code>`ifdef</code> / <code>`else</code> / <code>`endif</code>	Conditional compilation of code sections
<code>`undef</code>	Remove a previously defined macro

- ◆ Compiler directives begin with a back-tick (``) and are processed before actual compilation — similar to a C preprocessor.
- ◆ A directive remains active across all files compiled afterward, until undefined or the compilation ends.
- ◆ ``timescale` is essential for meaningful delay-based simulation results.
- ◆ Widely used to parameterize testbenches and manage conditional (debug vs synthesis) code.



System Tasks and Functions

Task / Function	Purpose
<code>\$display</code> / <code>\$write</code>	Print formatted output once (with / without newline). <code>\$display</code> is the main system task for displaying values of variables or strings or expressions. This is one of the most useful tasks in Verilog
<code>\$monitor</code>	Verilog provides a mechanism to monitor a signal when its value changes. Automatically print whenever any listed signal changes
<code>\$time</code> / <code>\$realtime</code>	Return current simulation time
<code>\$dumpfile</code> / <code>\$dumpvars</code>	Set up waveform (VCD) dumping for viewing in a waveform tool
<code>\$finish</code> / <code>\$stop</code>	End or pause/stop the simulation
<code>\$random</code>	Generate a pseudo-random number (useful in testbenches)

- ◆ Verilog provides standard system tasks for certain routine operations.
- ◆ System tasks/functions always begin with a dollar sign (\$), built directly into the Verilog language.
- ◆ All system tasks appear in the form `$<keyword>`.
- ◆ Operations such as displaying on the screen, monitoring values of nets, stopping, and finishing are done by system tasks.
- ◆ Primarily used inside testbenches for stimulus generation, monitoring and reporting — not synthesizable hardware.
- ◆ `$monitor` is especially useful for continuously tracking signal changes without repeated `$display` calls.



SECTION 5 OF 5

Simulation, Synthesis & Verification

Simulation vs synthesis, testbenches, file organization, and a hierarchical modeling example

Simulation vs Synthesis

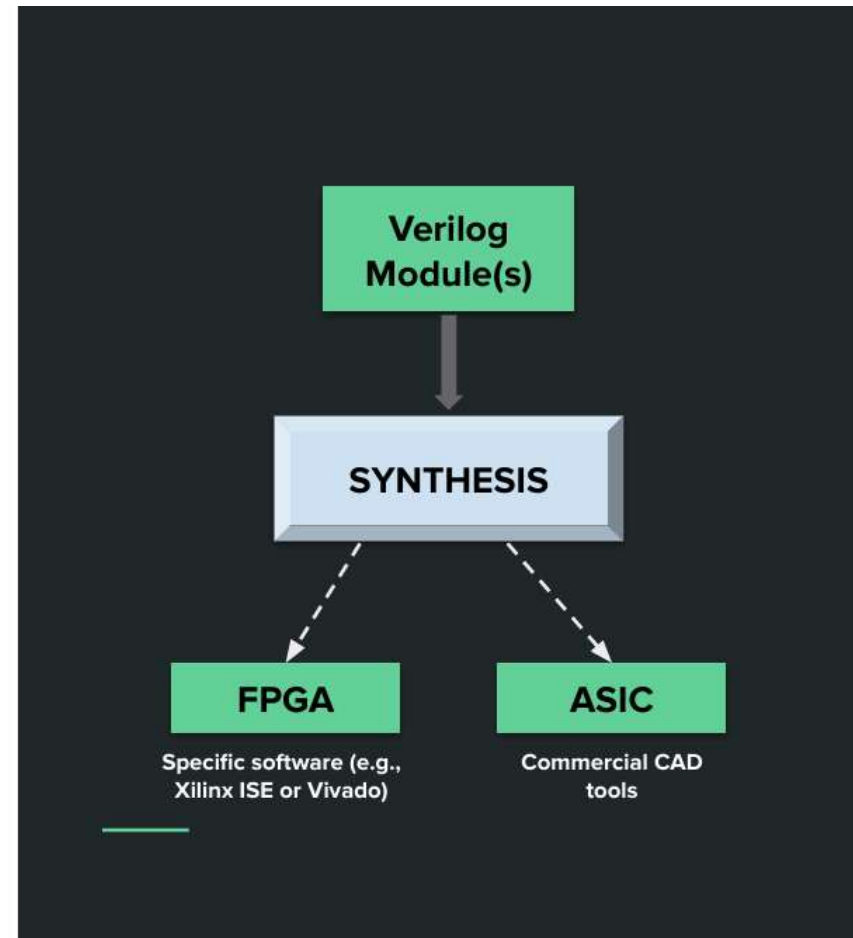
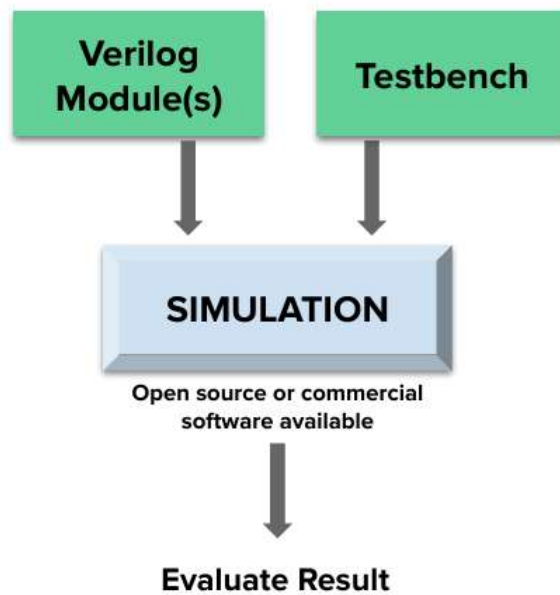
Simulation

- ✦ Executes Verilog code to verify functional correctness
- ✦ Runs on a simulator (e.g., ModelSim, Vivado, etc.)
- ✦ Uses testbenches, \$display / \$monitor for observation
- ✦ Supports non-synthesizable constructs (delays, file I/O)
- ✦ Output: pass/fail results, waveform (VCD) files

Synthesis

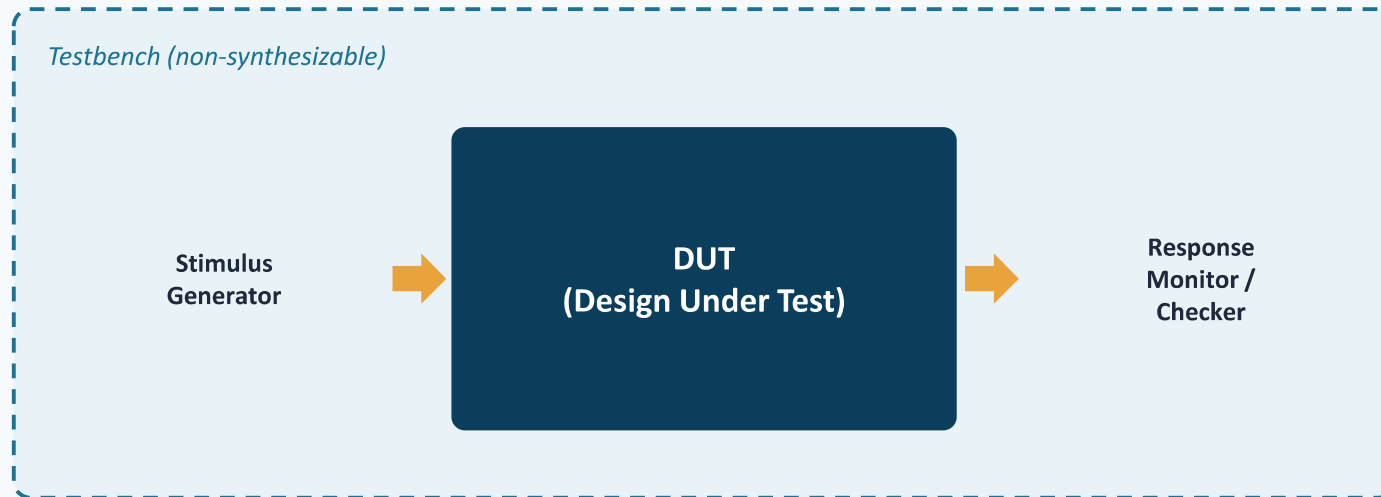
- ✦ Converts RTL Verilog into a gate-level netlist
- ✦ Runs on a synthesis tool (e.g., Yosys, Design Compiler)
- ✦ Maps design to a target technology library / FPGA fabric
- ✦ Only synthesizable constructs are allowed
- ✦ Output: gate-level netlist, timing & area reports

Development Process





Introduction to Testbench



- ◆ A testbench is a Verilog module (with no ports) that instantiates the DUT, applies stimulus, and checks its responses — written purely for simulation, never synthesized.
- ◆ Typically uses initial blocks, reg to drive DUT inputs, wire to observe DUT outputs, and system tasks (\$display, \$monitor) to report results.
- ◆ Good testbenches are self-checking — they compare actual vs expected output and flag mismatches automatically.



Design Source & Testbench File Organization

```
project_alu/
|-- rtl/                (design source files)
|   |-- alu_top.v
|   |-- adder.v
|   `-- mux.v
|-- tb/                (testbench files)
|   |-- alu_top_tb.v
|   `-- test_vectors.txt
|-- sim/                (simulation logs,
waveforms)
|   `-- alu_top.vcd
`-- scripts/            (compile / run scripts)
    `-- run_sim.do
```

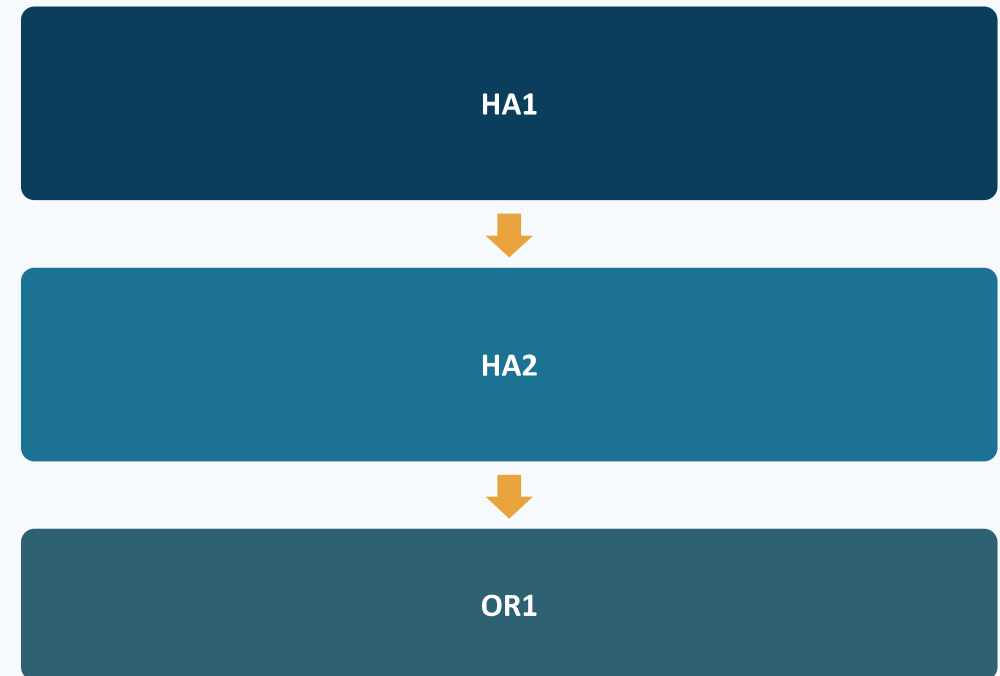
- ◆ Keeping design (RTL) and testbench files in separate directories avoids accidentally synthesizing testbench-only code.
- ◆ One module per file, with the file name matching the module name, is a common and NBA-friendly convention.
- ◆ A consistent structure (rtl / tb / sim / scripts) makes projects easier to version-control, review, and hand over between team members.
- ◆ Simulation outputs (waveforms, logs) are generated artifacts and should not be manually edited or mixed with source files.



Basic Example: Hierarchical Modeling

```
// Full adder built from two half adders + OR gate
module half_adder (sum, carry, a, b);
    output sum, carry;
    input  a, b;
    assign sum = a ^ b;
    assign carry = a & b;
endmodule

module full_adder (sum, cout, a, b, cin);
    output sum, cout;
    input  a, b, cin;
    wire s1, c1, c2;
    half_adder HA1 (s1, c1, a, b);
    half_adder HA2 (sum, c2, s1, cin);
    or OR1 (cout, c1, c2);
endmodule
```



full_adder (top module)



Few More Examples

```
// 4:1 MUX
module mux4to1_gates (i0,i1,i2,i3,s1,so,y);
    output y;
    input  i0,i1,i2,i3,s1,s0;
    wire s1n,s0n;
    wire a0,a1,a2,a3;
    not(s1n,s1);
    not (s0n, s0);
    and (a0, i0, s1n, s0n); // selects i0 when s1=0, s0=0
    and (a1, i1, s1n, s0); // selects i1 when s1=0, s0=1
    and (a2, i2, s1, s0n); // selects i2 when s1=1, s0=0
    and (a3, i3, s1, s0); // selects i3 when s1=1, s0=1
    or (y, a0, a1, a2, a3);
endmodule
```

```
//4:1 MUX Testbench
`timescale 1ns/1ps
module mux4to1_gates_tb;

    reg i0, i1, i2, i3;
    reg s1, s0;
    wire y;
    // Instantiate the DUT using positional mapping
    mux4to1_gates uut (i0, i1, i2, i3, s1, s0, y);
    initial begin
        i0 = 0; i1 = 1; i2 = 0; i3 = 1;
        s1 = 0; s0 = 0; #10; // expect y = i0 = 0
        s1 = 0; s0 = 1; #10; // expect y = i1 = 1
        s1 = 1; s0 = 0; #10; // expect y = i2 = 0
        s1 = 1; s0 = 1; #10; // expect y = i3 = 1
        $finish;
    end
    initial
    begin
        $monitor("time=%0t | s1=%b s0=%b | i0=%b i1=%b i2=%b
        i3=%b | y=%b", $time, s1, s0, i0, i1, i2, i3, y);
    end
endmodule
```



Key Takeaways

- ◆ Verilog HDL models digital hardware concurrently across behavioral, dataflow, gate and switch abstraction levels.
- ◆ A structured design flow — spec → RTL → simulation → synthesis → implementation — underlies both top-down and bottom-up methodologies.
- ◆ Modules with clear port interfaces (input / output / inout) are the building blocks of hierarchical, reusable designs.
- ◆ Compiler directives and system tasks support code organization, debugging, and testbench-driven verification.
- ◆ Simulation confirms correctness; synthesis converts verified RTL into real hardware — both stages are essential and distinct.
- ◆ Organizing RTL and testbench files separately, and practicing hierarchical modeling early, builds strong habits for student projects and labs.



Thank You

Questions & Discussion